



## PRIVILEGIOS DE ADMINISTRACIÓN, ROLES Y MECANISMOS DE AUTENTICACIÓN

### PRIVILEGIOS DE ADMINISTRACIÓN, ROLES Y MECANISMOS DE AUTENTICACIÓN..... 1

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| 1. 🎯 Objetivo.....                                                             | 2  |
| 2. 📖 Vistas del diccionario de datos asociadas con la base de datos.....       | 2  |
| 2.1. Revisión general del diccionario de datos.....                            | 2  |
| 2.2. Vistas dinámicas de desempeño del diccionario de datos.....               | 3  |
| 3. 😱 Escenario 1 - Pérdida del password de sys y del usuario oracle.....       | 4  |
| 4. 🔍 Escenario 2: Diagnóstico combinado de la instancia.....                   | 5  |
| 5. 🔑 Escenario 3 - Asignar roles para facilitar el control de privilegios..... | 7  |
| 6. 🛡 Escenario 4 - Delegar tareas administrativas a usuarios.....              | 8  |
| 7. 🗑 Escenario 5 - Pérdida y recuperación del archivo de passwords.....        | 10 |
| 8. ⚙ Escenario 6 - Diseño de roles y privilegios.....                          | 12 |
| 9. 🔧 Validador.....                                                            | 13 |
| 10. 📦 Contenido de la entrega.....                                             | 13 |

## 1. Objetivo

Comprender y poner en práctica los conceptos referentes a los privilegios de administración, así como los diferentes mecanismos de autenticación que pueden emplearse en una base de datos, aplicados a distintos escenarios que pueden ocurrir durante su operación.

 **Sobre el uso de la IA en esta práctica:** antes de consultar una herramienta de IA para resolver un escenario, se recomienda formular primero una hipótesis propia sobre la causa o la solución esperada. La evaluación de esta práctica considera esa justificación, no únicamente la entrega de un script funcional.

- Generar el siguiente directorio para guardar los scripts de esta práctica

```
mkdir -p ${UNAM_HOME}/bda/practicas/04
```

- Asignar permisos al usuario `oracle` para que pueda escribir en la carpeta `04`

```
chmod 777 ${UNAM_HOME}/bda/practicas/04
```

- Iniciar servicios. Para todas las prácticas del curso, no olvidar levantar los servicios para que la CDB pueda operar correctamente:
  - A. Levantar listener
  - B. Iniciar la instancia

## 2. Vistas del diccionario de datos asociadas con la base de datos

### 2.1. Revisión general del diccionario de datos

Una de las partes más importantes de una base de datos es su diccionario. El diccionario está formado por un conjunto de tablas base y vistas que proporcionan metadatos para realizar la administración de la propia base de datos.

- *Tablas base:* Almacenan información acerca de la base de datos. Únicamente el propio manejador lee y escribe en estas tablas. El formato de estas tablas es poco amigable para ser accedidas por usuarios comunes.
- *Vistas:* Decodifican las complejidades de las tablas bases en un conjunto de vistas con información útil para los usuarios.

Típicamente las vistas del diccionario de datos están agrupadas en conjuntos. Cada uno de estos conjuntos a su vez está formado por 4 tipos de vistas principales que se distinguen por el prefijo: `cdb`, `dba`, `user` y `all`. Estos 4 grupos contienen prácticamente la misma información la cual es accesible dependiendo el tipo de usuario.

| Prefijo            | Acceso a Usuarios                                                                                                                                                                     | Contenido                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>DBA_</code>  | Accesibles para usuarios con privilegios de administración <code>sysdba</code>                                                                                                        | Todos los objetos existentes a nivel PDB. Por ejemplo, la vista <code>dba_users</code> contendrá la lista de todos los usuarios existentes en la PDB donde se ejecuta la consulta.                                                                                                                                                                                                                                                                                                                                                                  |
| <code>ALL_</code>  | Todos los usuarios                                                                                                                                                                    | Todos los objetos a los que el usuario tiene privilegios de consulta o modificación. Por ejemplo, <code>all_tables</code> muestra las tablas propias del usuario y adicionalmente las tablas de otros usuarios a las que tiene acceso                                                                                                                                                                                                                                                                                                               |
| <code>USER_</code> | Todos los usuarios                                                                                                                                                                    | Todos los usuarios   Todos los objetos que le pertenecen al usuario en sesión. Por ejemplo, <code>user_tables</code> muestra únicamente las tablas propias del usuario que realiza la consulta                                                                                                                                                                                                                                                                                                                                                      |
| <code>CDB_</code>  | Para cada vista con el prefijo <code>DBA_</code> existirá una vista con el prefijo <code>CDB_</code> , accesibles para usuarios con privilegios de administración <code>sysdba</code> | Equivalente a <code>dba_</code> , pero mostrando objetos de toda la cdb, no solo de la pdb en la que se ejecuta la consulta. Incluye la columna <code>con_id</code> , que indica el contenedor donde se encuentra el objeto. Por ejemplo, si se realiza una consulta a <code>cdb_users</code> desde <code>cdb\$root</code> , se muestran los usuarios que existen en <code>cdb\$root</code> y en todas las PDBs. Si la consulta se realiza en una PDB, sólo se muestran los objetos de la PDB en cuestión (similar a las vistas <code>dba</code> ). |

■ No todos los conjuntos de vistas cuentan con estos 4 subgrupos.

Existe una vista muy particular llamada `dictionary` la cual contiene la lista y una breve descripción de **la gran mayoría** de las vistas del diccionario de datos. La columna `table_name` contiene el nombre de la vista. Uno de sus usos principales es la búsqueda de vistas que pudieran contener cierta información de interés.

```
sys@jrcbda1> desc dictionary
Name          Null?     Type
-----
TABLE_NAME    VARCHAR2(128)
COMMENTS      VARCHAR2(4000)
```

## 2.2. Vistas dinámicas de desempeño del diccionario de datos

Durante su operación, la base de datos mantiene un conjunto de tablas virtuales que almacenan información de su actividad. Se les conoce como vistas dinámicas debido a que son continuamente actualizadas durante la operación de la propia BD. Este tipo de vistas inician con el prefijo `V$`. La información que generalmente contiene este tipo de vistas es:

- Parámetros de sesión y del sistema
- Uso de memoria
- Estado o estatus de los archivos de la base de datos.
- Progreso de tareas programadas.
- Ejecución de sentencias SQL
- Estadísticas y métricas.

### 3. 🤖 Escenario 1 - Pérdida del password de sys y del usuario oracle

Suponer que no se conoce el password del usuario `sys`, ni el password del usuario del sistema operativo `oracle`. Solo se cuenta con el usuario administrador del sistema, que puede hacer uso del comando `sudo`.

- ¿Qué problemas existirán al no contar con ninguno de los dos passwords?
- ¿Cómo se podría recuperar el acceso administrativo en este escenario, sin conocer el password de `oracle`?

Crear un script llamado `s-01-sys-pass-admin.sh`, ejecutado por el usuario administrador (distinto de `oracle` y de `root`), que realice lo siguiente:

- Emplear `sudo` para ejecutar `sqlplus` como el usuario `oracle`, sin necesidad de conocer su password.
- Autenticar por sistema operativo (sin proporcionar usuario/password de base de datos).
- Actualizar el password de `sys` al valor `system1_p4`.
- Verificar la actualización conectando con el archivo de passwords, empleando el nuevo valor, tanto en `cdb$root` como en la pdb `<iniciales>bda_s1`.

Completar los **TODO** del siguiente script base:

```
#!/bin/bash
# @Autor      <Nombre del alumno>
# @Fecha      dd/mm/yyyy
# @Descripción  Recuperación del password de sys mediante autenticación por sistema
#              operativo a través de sudo, sin conocer los passwords de sys ni de
#              oracle. Ejecutado por el usuario administrador.

usuarioOs=$(whoami)

if [ "${usuarioOs}" = "oracle" ] || [ "${usuarioOs}" = "root" ]; then
    echo "❌ ERROR: el script debe ser ejecutado por el usuario administrador (no oracle, no root)"
    exit 1
fi

echo "🔧 Actualizando el password de sys mediante autenticación por s.o. (oracle via sudo)"

# Las opciones -u y -i de sudo permiten ejecutar sqlplus como el usuario oracle.
# Esto es posible porque el usuario administrador tiene permisos
# de sudo para ejecutar comandos como oracle.
```

```

# Notar la sintaxis <<EOF ... EOF empleada para pasar múltiples líneas de
# comandos a sqlplus. Es importante que el EOF de cierre esté al inicio de la
# línea, sin espacios ni tabulaciones.
# La instrucción "whenever sqlerror exit sql.sqlcode" indica a sqlplus que
# salga con el código de error de SQL si ocurre algún error
# durante la ejecución de los comandos.
sudo -u oracle -i sqlplus -s /nolog <<EOF
  whenever sqlerror exit sql.sqlcode
  -- TODO: Autenticar por sistema operativo como sysdba (comando connect,
  -- TODO: sin usuario ni password explícitos),
  -- TODO: actualizar el password de sys al valor system1_p4

  ----- Respuesta -----
  -----

  prompt  Password de sys actualizado correctamente
  exit
EOF

# El operador $? contiene el código de salida del último comando ejecutado.
# Un valor distinto de 0 indica que ocurrió un error.
if [ $? -ne 0 ]; then
  echo " ERROR: no fue posible actualizar el password de sys"
  exit 1
fi

echo " Probando conexión con el nuevo password en cdb\${root}"

# Verificar la conexión autenticando con el archivo de passwords, empleando el
# usuario sys y el nuevo password (system1_p4). En este caso ya no es necesario usar
# sudo, ya que el usuario administrador puede conectarse con sysdba mediante el
# archivo de passwords
sqlplus -s /nolog <<EOF
  whenever sqlerror exit sql.sqlcode
  --TODO Agregar la instrucción connect para autenticar con el nuevo password
  --del usuario sys en cdb\${root}

  ----- Respuesta -----
  -----

  select user from dual;
  prompt  Conexión con el nuevo password en cdb\${root} exitosa
  exit
EOF

if [ $? -ne 0 ]; then
  echo " ERROR: no fue posible autenticar con el nuevo password en cdb\${root}"
  exit 1
fi

echo " Probando conexión con el nuevo password en <iniciales>bda_s1"

```

```

sqlplus -s /nolog <<EOF
  whenever sqlerror exit sql.sqlcode
  --TODO Agregar la instrucción connect para autenticar con el nuevo password
  --del usuario sys en <iniciales>bda_s1

  ----- Respuesta -----
  -----

  select user from dual;
  prompt  Conexión con el nuevo password en <iniciales>bda_s1 exitosa
  exit
EOF

if [ $? -ne 0 ]; then
  echo "❌ ERROR: no fue posible autenticar con el nuevo password en <iniciales>bda_s1"
  exit 1
fi
echo ""
echo "🏆 OK. El password de sys fue actualizado y verificado correctamente"

```

Ejecutar el script, verificar que no se hayan presentado errores.

#### 4. Escenario 2: Diagnóstico combinado de la instancia

Parte de las tareas de un DBA es realizar consultas frecuentes al diccionario de datos para obtener información del funcionamiento de la base de datos. Para revisar y evaluar estas consultas, los datos obtenidos serán almacenados en tablas propias de un usuario ordinario, de forma que puedan ser consultadas posteriormente sin necesidad de que el validador se conecte como `sys` y al mismo tiempo, podrán ser revisadas por el validador de cada práctica.

Para implementar esta técnica, generalmente estas consultas las realiza el usuario `sys`, y el resultado se almacena en una tabla propia de un usuario ordinario. Ejemplo:

```

drop table if exists <esquema_usuario>.<nombre_tabla>;
create table <esquema_usuario>.<nombre_tabla> as
select c1,c2,... from <vistas_diccionario> where <condición>;

```

 Notar el uso de la instrucción `drop table if exists`. Esta característica permite implementar scripts llamados **idempotentes**, es decir, que pueden ejecutarse varias veces sin que se generen errores y producen exactamente el mismo resultado. Esta técnica es muy útil para scripts de diagnóstico, pruebas, etc. Permite ejecutar el mismo script varias veces sin preocuparse por el estado previo del entorno. A lo largo del curso procurar que todos los scripts que se generen sean idempotentes.

Para familiarizarse con esta técnica, crear un script llamado `s-02-diagnostico-bd.sql` que muestre información de la versión de la CDB así como datos de la instancia, a partir de la consulta de 2 vistas del diccionario de datos. El script deberá realizar las siguientes acciones:

- A. Conectar como `sysdba`.
- B. Crear un usuario llamado `<nombre>0401` (idempotente: `drop user if exists <nombre>0401 cascade;` antes de crearlo). Asignar privilegios para crear sesiones y tablas, con cuota ilimitada en el tablespace `users`.
- C. Para determinar la vista que contiene el nombre de la instancia, empleando la vista `dictionary` (ver *Revisión general del diccionario de datos*), localizar una vista **dinámica** (`v$`, ver *Vistas dinámicas de desempeño del diccionario de datos*) que contenga el nombre y el instante de arranque de la instancia en ejecución.
- D. Para obtener el detalle de la versión de la CDB considerar la vista `product_component_version`.

 **Pista:** Para determinar el nombre de la vista adecuada, consultar `dictionary` filtrando por palabras clave como ayuda para acotar las vistas candidatas antes de decidir cuál usar, por ejemplo:

```
set linesize window
col table_name format a30
col comments format a50
select table_name, comments
from dictionary
where table_name like 'V$%INSTANCE%';
```

También se recomienda consultar el detalle de la vista empleando la documentación oficial de Oracle, para identificar las columnas que contienen la información requerida. En el navegador buscar: `<nombre_vista>` Oracle `<version>`

- E. Construir una única consulta que combine ambas vistas junto con el identificador de contenedor (`con_id`) desde el cual se ejecuta la consulta, y almacenar su resultado en una tabla `t01_diagnostico` perteneciente al esquema del usuario `<nombre>0401`, con las siguientes columnas: `product`, `version_full`, `instance_name`, `startup_time`, `con_id`.

 **Pista:** ambas vistas fuente devuelven un único renglón, por lo que se puede hacer un `join` cruzado (cross join) entre ellas. El identificador de contenedor se obtiene a través de una función de contexto de sesión (`sys_context`), no de una columna de alguna de estas vistas. Estructura general a seguir (pseudo-código):

```
drop table if exists <esquema_usuario>.t01_diagnostico;
create table <esquema_usuario>.t01_diagnostico as
select p.product,p.version_full,v.instance_name,
       to_char(v.startup_time,'dd-mm-yyyy hh24:mi:ss') as startup_time,
       sys_context('userenv','con_id') as con_id
from <vista_1> p, <vista_2> v;
```

## 5. Escenario 3 - Asignar roles para facilitar el control de privilegios

Un rol es un objeto cuyo objetivo es agrupar un conjunto de privilegios. Un rol puede asignarse a varios usuarios, por lo que la lista de privilegios asociada al rol será otorgada a cada uno de ellos.

```
create role <nombre_rol>;
grant <privilegio_1>, <privilegio_2>, ..., <privilegio_N> to <nombre_rol>;
```

Esto simplifica la administración de privilegios sobre un conjunto de usuarios. Por ejemplo, suponer que en una base de datos, a todos los usuarios que sean desarrolladores `dev_01`, `dev_02`, ..., `dev_N` se les debe otorgar una lista de privilegios `p1`, `p2`, ..., `pN`. En lugar de asignarlos uno a uno para cada usuario, se puede crear un rol `dev_rol` que agrupe estos privilegios, y asignar dicho rol a cada desarrollador.

```
grant dev_rol to <dev_01>, <dev_02>, ..., <dev_N>;
```

Si en el futuro se decide agregar o quitar un privilegio para los desarrolladores, basta con modificarlo en el rol. El cambio se refleja automáticamente en todos los usuarios que lo tienen asignado.

```
grant <privilegio_extra> to dev_rol;
revoke <privilegio_a_quitar> from dev_rol;
```

Para poner en práctica estos conceptos, crear un script llamado `s-03-roles.sql` que realice las siguientes acciones:

- Conectar como `sysdba` en la pdb `<iniciales>bda_s1`.
- Crear 2 usuarios `<nombre>04_dev_01` y `<nombre>04_dev_02` (idempotente: `drop user if exists <usuario> cascade`; antes de cada uno). Asignar como password el valor `<nombre>`.
- Estos 2 usuarios deben tener privilegios para crear sesión, tablas, vistas y procedimientos. Para evitar otorgarlos individualmente a cada usuario, agrupar estos 4 privilegios en un rol llamado `p04_dev_role`, y asignar dicho rol a ambos usuarios. Antes de crear el rol, verificar que no exista previamente y eliminarlo si es necesario (`drop role if exists p04_dev_role`).

- D. Suponer que, tiempo después, se decide hacer los siguientes ajustes a los privilegios de estos usuarios: quitarles el privilegio para crear procedimientos, y agregarles el privilegio para crear secuencias. Realizar el ajuste directamente sobre el rol, de forma que el cambio se refleje automáticamente para ambos usuarios.
- E. Para comprobar los resultados, realizar la siguiente consulta al diccionario de datos, que muestra los privilegios asignados a cada usuario. Comprobar que solo se obtienen privilegios para crear tablas, vistas y secuencias.

```
col grantee format a30
col granted_role format a50
set linesize window
select grantee, granted_role
from dba_role_privs
where grantee in ('<NOMBRE>04_DEV_01', '<NOMBRE>04_DEV_02')
order by 1, 2;
```

 **Nota:** los nombres de objetos como tablas, vistas y usuarios se guardan en el diccionario en mayúsculas. Por esta razón, los nombres de usuario en la consulta anterior se escriben en mayúsculas.

- F. Ejecutar el script con el usuario administrador.

## 6. Escenario 4 - Delegar tareas administrativas a usuarios

Suponer que se tiene una CDB cuyas tareas administrativas las realizarán los siguientes usuarios con sus respectivos privilegios de administración

- Usuario `<nombre>0402` se le otorga el privilegio `sysdba`
- Usuario `<nombre>0403` se le otorga el privilegio `sysoper`
- Usuario `<nombre>0404` se le otorga el privilegio `sysbackup`

Lo anterior implica que estos 3 usuarios podrán entrar a sesión ya sea como usuarios ordinarios o como usuarios administradores.

Suponer el siguiente requerimiento: Al final de cada día, los 3 usuarios anteriores deberán registrar el número de horas que trabajaron en el día. Deberán generar 2 registros:

- Número de horas que trabajaron como usuarios ordinarios
- Número de horas que trabajaron como usuarios administradores.

La tabla donde deberán insertar sus registros le pertenece a un usuario llamado `<nombre>04_admin`.

Crear un script `s-04-privs-admin.sql` para realizar las siguientes tareas:

- A. Conectarse como `sysdba`
- B. Crear a los 3 usuarios. Asignar como password el valor `<nombre>`.
- C. Realizar las configuraciones necesarias para que los usuarios puedan crear sesiones.
- D. Asignar sus privilegios de administración.
- E. Crear al usuario `<nombre>04_admin` con cuota de almacenamiento ilimitada en el tablespace `users`, con privilegios para crear sesiones y tablas

F. Crear la tabla `t01_bitacora` la cual le pertenece al usuario `<nombre>04_admin`.

```
drop table if exists <nombre>04_admin.t01_bitacora;
create table <nombre>04_admin.t01_bitacora(
  id number not null,
  usuario varchar2(30) not null,
  esquema varchar2(30) not null,
  rol varchar2(20) not null,
  fecha_registro date not null
);
```

G. Para que los 3 usuarios puedan insertar sus registros en esta tabla, ya sea como usuario ordinario o como usuario administrador, en algunos casos se requiere que el usuario `<nombre>04_admin` o en su defecto el usuario `sys`, les otorgue privilegios para hacer inserciones. Revisar y completar la siguiente tabla para verificar en qué casos se requiere otorgar permisos.

| # | usuario                         | Esquema asignado                | privilegio/usuario con el que se autentica | ¿requiere permisos para insertar? |
|---|---------------------------------|---------------------------------|--------------------------------------------|-----------------------------------|
| 1 | <code>&lt;nombre&gt;0402</code> | <code>&lt;nombre&gt;0402</code> | <code>ordinario</code>                     | SI                                |
| 2 | <code>sys</code>                | <code>sys</code>                | <code>sysdba</code>                        | NO                                |
| 3 | <code>&lt;nombre&gt;0403</code> |                                 | <code>ordinario</code>                     |                                   |
| 4 |                                 |                                 | <code>sysoper</code>                       |                                   |
| 5 | <code>&lt;nombre&gt;0404</code> |                                 | <code>ordinario</code>                     |                                   |
| 6 |                                 |                                 | <code>sysbackup</code>                     |                                   |

La tabla con los valores faltantes será **incluida en la entrega**. Una vez que se haya completado la tabla, generar las sentencias SQL necesarias para otorgar privilegios de inserción a la tabla. Por ejemplo, para que el usuario `<nombre>0402` pueda insertar. Para ello conectar como usuario `<nombre>04_admin` y otorgar privilegios de inserción sobre su tabla.

```
connect <nombre>04_admin/<nombre>@<iniciales>bda_s1
grant insert on t01_bitacora to <nombre>0402;
--Completar para Los demás casos que requieran permisos para insertar
```

H. Para comprobar las configuraciones anteriores, realizar 6 conexiones (una para cada renglón de la tabla) y realizar la inserción correspondiente. Por ejemplo, para el primer renglón:

```
--renglón 1
```

```
connect <nombre>0402/<nombre>@<iniciales>bda_s1
insert into <nombre>04_admin.t01_bitacora(
  id, usuario,esquema,rol,fecha_registro)
values( 1,
  sys_context('USERENV','CURRENT_USER'),
  sys_context('USERENV','CURRENT_SCHEMA'),
  'ordinario',
  sysdate
);
--completar
```

- I. Ejecutar el script con el usuario administrador.

## 7. Escenario 5 - Pérdida y recuperación del archivo de passwords

Suponer que en este punto ocurre una falla y se pierde el archivo de passwords. Esta pérdida provocará que ningún usuario de administración podrá autenticar con este mecanismo.

¿Qué sucederá con los 3 usuarios a los que se les asignó su privilegio de administración? Antes de continuar, predecir la respuesta con base en lo revisado hasta este punto de la práctica. Para simular la pérdida del archivo y realizar su recuperación, se considera la siguiente estrategia:

- Respaldar el archivo de password existente
- Borrar el archivo de passwords
- Generarlo nuevamente
- Realizar configuraciones adicionales para que su contenido sea idéntico al original.

Generar un shell script llamado `s-05-archivo-passwords-oracle.sh` Tomar como base al siguiente código, leer con cuidado y completar donde se indica. Notar que el script debe ser ejecutado por el usuario `oracle`.

```
#!/bin/bash
# @Autor      Jorge Rodriguez
# @Fecha      dd/mm/yyyy
# @Descripcion Simulación de pérdida y recuperación de un archivo de passwords
archivoPwd="${ORACLE_HOME}/dbs/orapwfree"
practicaDir="/unam/bda/practicas/04"
archivoRespaldo="${practicaDir}/orapwfree.backup"

if [ "${USER}" != "oracle" ]; then
  echo "ERROR: el script debe ser ejecutado por el usuario oracle"
  exit 1
fi;
```

```
echo "Verificando si el archivo ya fue respaldado"
if [ -f "${archivoRespaldo}" ]; then
    echo "El archivo ${archivoRespaldo} ya existe, se omite su copia"
else
    echo "Realizando respaldo del archivo de passwords, indicar password de ${USER}"
    cp ${archivoPwd} ${archivoRespaldo}
fi;

echo "Validando si el respaldo se realizó de forma correcta"
#TODO: similar a la validación anterior, agregar un if-else que verifique
#que el archivo de respaldo existe. De lo contrario, imprimir ERROR y salir.

echo "Borrando el archivo de passwords original"
rm -f ${archivoPwd}

echo "Verificando que el archivo haya sido realmente eliminado"
#TODO: similar a la validación anterior, agregar un if-else que verifique
#que el archivo original se ha borrado. De lo contrario, imprimir ERROR y salir.

#En esta instrucción, el programa entra en modo de espera hasta que se presione
#cualquier tecla para continuar con la ejecución. Solo se agrega para confirmar
#que el archivo se ha borrado y se procede ahora a regenerar el archivo.
read -p "OK, archivo de passwords fue eliminado [Enter] para realizar su recuperación"

echo "Generando un archivo de passwords nuevo, proporcionar el password de SYS a Hola1234*"
#TODO: Agregar la instrucción para generar el archivo de passwords. Incluir
#únicamente al usuario SYS. El comando deberá incluir al password Hola1234* para
#evitar que lo pregunte al ejecutar este script

echo "validando la existencia del nuevo archivo"
if [ -f "${archivoPwd}" ]; then
    echo "OK. Archivo de password generado"
else
    echo "ERROR: El archivo de passwords no ha sido regenerado"
    exit 1
fi;
```

A. Ejecutar el script con el usuario `oracle`.

Hasta este punto, el archivo de passwords fue recuperado. Sin embargo, el archivo solo contiene al usuario `sys`, y tiene un password diferente al original. Se requieren realizar cambios. Crear un script `s-06-actualiza-archivo-passwords.sql` que realice las siguientes tareas:

- A. Actualizar el password de `sys` a su valor original, es decir, con el valor asignado desde su creación, por ejemplo, `system1`.
- B. Debido a la pérdida, los 3 usuarios creados anteriormente no podrán autenticar con su privilegio de autenticación. Para resolver este inconveniente, volver a asignar sus privilegios. Esto provocará la actualización del archivo.
- C. Realizar una consulta a la vista `v$pwfile_users` para comprobar que el archivo tiene 4 usuarios. `sys`, y los 3 usuarios creados anteriormente con su respectivo privilegio.

```
col username format a20
select username,sysdba,sysoper,sysbackup,syskm,sysdgm,con_id
from v$pwfile_users;
```

## 8. Escenario 6 - Diseño de roles y privilegios

Suponer el siguiente caso: un equipo de trabajo está formado por 3 dba junior, 2 desarrolladores, y 1 auditor de solo lectura. Cada perfil requiere un conjunto distinto de privilegios sobre la base de datos, y se espera que el equipo crezca con el tiempo.

- ¿Qué roles se propondrían para este equipo, y qué privilegios agruparía cada uno?
- ¿Por qué conviene agrupar los privilegios de esta forma, en vez de asignarlos individualmente a cada integrante?
- ¿Qué ventaja ofrece este diseño ante el crecimiento futuro del equipo mencionado en el escenario?

 **Nota:** este escenario no requiere la creación de un script — la respuesta se documenta en la hoja de cálculo de la práctica, ya que evalúa un criterio de diseño que no puede verificarse de forma programática.

## 9. Validador

- A. Obtener todos los archivos de la carpeta compartida correspondiente a esta práctica
- B. Cambiar permisos de ejecución a los shell scripts
- C. Abrir el archivo `sv-08-main.sql` cambiar los valores de las variables al inicio del script según corresponda.
- D. Ejecutar el validador empleando al usuario administrador. Notar que el validador va a ejecutar todos los scripts de la práctica incluyendo el script de limpieza.

```
su -l jorge
chmod 755 *.sh
sqlplus /nolog
start sv-08-main.sql
```

- E. El comando anterior va a generar un script llamado `p<nn>-sql-output.txt` . Este script deberá ser incluido en la entrega.

 **Nota:** dado que los scripts de la práctica son idempotentes (ver la nota de diseño del Escenario 2), el validador puede ejecutarse más de una vez sin necesidad de limpiar objetos previamente.

## 10. Contenido de la entrega

- Contestar la hoja de respuestas de la práctica, incluir los elementos solicitados.
- Incluir los scripts solicitados.
- Incluir el archivo de salida `p<nn>-sql-output.txt`